Group Project: Optimal Binary Search Tree

Source comparison: the Canvas assignment text (professor) vs. Problem 40 in the textbook (Jeff Erickson, *Algorithms*, Chapter 3 "Dynamic Programming," Exercises).

Due: Fri 10/9/2026, 11:59pm MT | **Canvas:** snow.instructure.com/courses/1254074/assignments/19231864 | **Total:** 25 pts

How this PDF is organized. Section 1 is the book problem exactly as printed (page scans, plus a typed transcription). Section 2 is the Canvas text and rubric, copied verbatim. Section 3 lines up each claim in the professor's summary against the book text so you can see what was restated, what was added, and what differs. The professor's provided test code is included in Section 2 and clearly labeled as the professor's.

1. The book: Problem 40 (verbatim)

Erickson, *Algorithms*, Ch. 3, Exercises (jeffe.cs.illinois.edu/teaching/algorithms/book/03-dynprog.pdf, PDF pages 50-51). Scans first; typed copy follows. The mark before "40" is Erickson's marker for a harder problem.

♠40. Congratulations! You've been hired by the giant online bookstore DeNile ("Not just a river in Egypt!") to optimize their warehouse robots. Each book that DeNile sells has a unique ISBN (International Standard Book Number), which is just a numerical value. Each of DeNile's warehouses contains a long row of bins, each containing multiple copies of a single book. These bins are arranged in sorted order by ISBN; each bin's ISBN is printed on the front of the bin in machine-readable form. Books are retrieved from these bins by robots, which run along rails parallel to the row of bins.

DeNile does not maintain a list of which bins contain which ISBN numbers; that would be too simple! Instead, to retrieve a desired book, the robot must first find that book's bin using a binary search. Because the search requires physical motion by the robot, we can no longer assume that each step of the binary search requires $O(1)$ time. Specifically:

- The robot always starts at the "0th bin" (where the books are loaded into boxes to ship to customers).
- Moving the robot from the ith bin to the jth bin requires $\alpha|i-j|$ seconds for some constant α.
- The robot must be directly in front of a bin in order to read that bin's ISBN. Reading an ISBN requires β seconds, for some constant β.
- Reversing the robot's direction of motion (from increasing to decreasing or vice versa) requires γ additional seconds, for some constant γ.
- When the robot finds the target bin, it extracts one book from that bin and returns to "the 0th bin".

Design and analyze an algorithm to compute a binary search tree over the bins that minimizes the total time the robot spends searching for books. Your input is an array $f[1..n]$ of integers, where $f[i]$ is the number of times that the robot will be asked to retrieve a book from the ith bin, along with the time parameters α, β, and γ.

Typed transcription of Problem 40

Congratulations! You've been hired by the giant online bookstore DeNile ("Not just a river in Egypt!") to optimize their warehouse robots. Each book that DeNile sells has a unique ISBN (International Standard Book Number), which is just a numerical value. Each of DeNile's warehouses contains a long row of bins, each containing multiple copies of a single book. These bins are arranged in sorted order by ISBN; each bin's ISBN is printed on the front of the bin in machine-readable form. Books are retrieved from these bins by robots, which run along rails parallel to the row of bins.

DeNile does not maintain a list of which bins contain which ISBN numbers; that would be too simple! Instead, to retrieve a desired book, the robot must first find that book's bin using a binary search. Because the search requires physical motion by the robot, we can no longer assume that each step of the binary search requires O(1) time. Specifically:

- The robot always starts at the "0th bin" (where the books are loaded into boxes to ship to customers).
- Moving the robot from the ith bin to the jth bin requires $\alpha|i - j|$ seconds for some constant α.
- The robot must be directly in front of a bin in order to read that bin's ISBN. Reading an ISBN requires β seconds, for some constant β.
- Reversing the robot's direction of motion (from increasing to decreasing or vice versa) requires γ additional seconds, for some constant γ.
- When the robot finds the target bin, it extracts one book from that bin and returns to "the 0th bin".

Design and analyze an algorithm to compute a binary search tree over the bins that minimizes the total time the robot spends searching for books. Your input is an array $f[1..n]$ of integers, where $f[i]$ is the number of times that the robot will be asked to retrieve a book from the ith bin, along with the time parameters α, β, and γ.

Note: the PDF's text layer garbles the Greek letters, so α, β, γ in this typed copy were restored by hand. The scans above are the authoritative copy.

2. Canvas assignment text and rubric (verbatim)

Copied from *advanced algorithms/assignment information/Group Project - Optimal Binary Search Tree.md* (synced from Canvas 2026-10-03). It includes the professor's provided test code, which is boxed and labeled below.

With a partner, implement and test a top-down dynamic programming solution to Problem 40 from chapter 3 of Jeff Erickson's online textbook (with the additional details and clarifications below). Whatever you submit should be something that you can recreate on paper without other resources (other than basic notes on Python language features).

As usual, use your non-AI-enabled Classwork Profile for this assignment (no AI code generation or completion).

Summary of the Problem and Clarifications

The problem describes a library where books are fetched by a robot.

Books are stored in bins ordered by increasing ISBN---one bin per owned book title, and books are requested, one copy at a time, by submitting the ISBN to the robot, which then follows a binary search tree over the bin indexes to locate the bin with the matching ISBN.

Based on a variety of historical data and current information, the library has a guess as to the expected number of times each bin will be accessed on any particular day, so the binary search can be re-programmed each day to try to minimize total access time over the day. An array of expected bin frequencies for the day are given as part of the problem input.

The robot must start out at bin 0, facing in the "negative" direction (toward the front desk) to receive an ISBN request.

After receiving the ISBN, the robot must move to the root of the pre-programmed binary search tree, but the robot can only move in the direction it is facing. Whenever the robot must change directions it incurs a cost, and the cost_per_rotation is given as part of the problem input.

If facing in the correct direction, the robot can move to a target index which incurs an additional movement cost which is proportional to the distance that the robot must move. The move_cost_per_distance is given as part of the problem input.

Once the robot arrives at the index corresponding to the tree root, it scans the QR-code on the bin which lets the robot know whether the target ISBN is less than, equal to, or greater than the scanned bin. The robot can scan while facing in the negative or positive direction. The cost_per_scan is given as part of the problem input.

If the robot discovers that the scanned bin has the target ISBN, the robot ensures that it is facing in the negative direction and then moves back to index 0 to deliver the book. Otherwise, the robot consults the appropriate left or right child index stored in the pre-programmed binary search tree and prepares to move to that index to continue the search.

Ultimately, the top-level problem is to efficiently find, given a list of item expected frequencies and the three cost parameters, the binary search tree (over bin indexes) that minimizes the total expected cost of all book deliveries during the day. You will build up to this in four main parts, plus a final submission part:

- Describe the recursive structure of the problem in words/math
- Use backtracking only (no dynamic programming) to compute the minimum possible cost (do not bother to build up the actual tree yet)
- Add dynamic programming (via memoization with lru_cache or cache) to compute the minimum possible cost efficiently
- Add the bookkeeping necessary to efficiently construct the optimal tree in addition to computing the optimal cost

Part 1 really represents the bulk of the work, but until you put it into correct code, you might not realize that your solution has holes in it, which is why the concrete implementation can be helpful. I also hope that you can appreciate the very fine line between the correct and slow backtracking implementation and the correct and fast implementation that uses memoization.

Part 1 -- Planning your solution

Work the first three provided test cases (below) out by hand and confirm you get the same numbers. If your hand calculation disagrees with the test, sort that out now -- everything after this depends on you having the cost model right.

After you are clear on the parameters and functionality of the top-level function, answer the following questions in your .py file using a combination of English and Math (you don't necessarily need to figure these out in this order):

- What is the best name that you can think of for the top-level function that only returns the optimal cost (not-the tree) and what parameters should the function accept and what does the function return? (Be as precise and clear as you can be.)
- How can you carve off part of the problem to be solved by a nested, recursive function, and what parameters would be necessary to identify a particular subproblem? Note: the recursive function doesn't need to be given any of the information already available in the top-level function since it already has access to it; rather, the nested function just needs to know what part of the problem it is working on and any context that would impact the cost of solving that part of the problem (e.g. where is the robot traveling from). Of course, since we will be using memoization with a hash table, make sure that you only use hashable parameter types to the nested function.
- How will the top-level function use the nested function to solve the top-level problem. (I.e. What nested function calls will the top-level function make to solve the top-level problem?)
- (a) What instances of the nested problem are base cases? and (b) What should the nested function return for each base case?
- (a) What part of the solution will the nested function directly choose (hint: what should be the root index for the problem subset)? (b) What options will it consider for that choice? (c) How will it determine the cost/value of each candidate (including what recursive calls it will need to make as part of that calculation and how will those results be combined)? and (d) How will it decide how to combine the results for the various options (hint: in this case, you want the option with the smallest cost)?

Part 2 -- Min Cost Backtracking Search

Write optimal_cost_backtracking so that it passes the provided test_backtracking_finds_the_optimal_cost cases, without memoization or any other dynamic programming technique. Regardless of the name you came up with in Part 1, use the names the tests use here: the function optimal_cost_backtracking with parameters frequencies, cost_per_rotation, move_cost_per_distance, and cost_per_scan (the tests pass the costs by name).

Part 3 -- Min Cost Dynamic Programming Search

Copy your part 2 function to optimal_cost_memoized (same parameter names) and add dynamic programming via memoization with the functools.lru_cache or functools.cache decorator on your nested recursive function. (You may need to raise the recursion limit via sys.setrecursionlimit.) It has to get the same answers as part 2 and also finish test_memoized_handles_fifty_bins, which your part 2 function would never finish.

Part 4 -- Return the Optimal Tree

Copy your part 3 function to optimal_tree (same parameter names) and add the bookkeeping to return the optimal binary search tree (as a nested tuple) alongside the minimum cost, as a (cost, tree) pair.

Hints:

When considering the cost of using a binary search tree on a span of bins, notice that every request for a bin in that span must first physically pass through the index at the root of the tree. So when you calculate the rotations, movement, and scan needed to check the root, you will need to know the total frequency of all the indexes in the span.

Nested tuples are great for representing trees—an empty tuple represents an empty tree and a (value, left_tree, right_tree) triple represents a non-empty tree. For this assignment, use the book indices (0, 1, 2, ...) as the values in the tree, not actual ISBNs. For example, the following tree has 10 at the root node, the left subtree has 7 at root and the right subtree has 12 at root, etc.; the node with value 8 doesn't have a left child (if you turn your head sideways to the right, you can kind of see the tree structure):

```
value, left, right = (10, (7, (3, (1, (0, (), ()),
                                   (2, (), ())),
                                (5, (4, (), ()),
                                   (6, (), ()))),
                            (8, (),
                                (9, (), ()))),
                      (12, (11, (), ()),
                           ()))
```

(Note that building the tree (value, left, right) can be done in constant time if left and right are already built since only the references are copied into the tuple rather than having the contents be deep-copied.)

PROFESSOR'S PROVIDED TEST CODE (not written by us). Everything from here to the matching "End" box is given by the professor on Canvas and must be pasted as-is at the bottom of the .py file.

Provided Tests

Each case with a tree listed has exactly one optimal tree, so there is a single right answer to compare against. If you want to create additional classes for your trees rather than just using tuples: make sure that (1) you don't change the big O space or running time and (2) make sure that you only make the minimal changes to the provided tests to accommodate your interface changes.

Tests -- paste these as-is at the bottom of your .py file (click to expand)

```
# (frequencies, rotation, move, scan, optimal cost, optimal tree)
# Each case has exactly one optimal tree, so the tree is safe to check.
OPTIMAL_BST_CASES = [
    ([5],                3, 1, 2,   10, (0, (), ())),
    ([4, 3],             3, 1, 2,   44, (0, (), (1, (), ()))),
    ([4, 30],            3, 1, 2,  348, (1, (0, (), ()), ())),
    ([1, 1, 1],          3, 1, 2,   30, (0, (), (1, (), (2, (), ()))),
    ([4, 3, 28, 100, 5], 3, 1, 2, 2072, (3, (2, (1, (0, (), ()), ()), ()), (4, (), ())),
    ([1, 1, 1, 1, 1, 50], 7, 2, 1, 1940,
     (5, (4, (3, (2, (1, (0, (), ()), ()), ()), ()), ()), ()), ()),
    ([3, 1, 4, 1, 5, 9, 2, 6], 5, 2, 3, 1132,
     (2, (1, (0, (), ()), ()),
         (5, (4, (3, (), ()), ()), (7, (6, (), ()), ()))),
    ([20, 5, 1, 1, 30, 2, 2, 40], 4, 1, 0, 1512,
     (0, (), (1, (), (2, (), (3, (), (4, (), (5, (), (6, (), (7, (), ((((((((((),
]

# Fifty bins. Only a memoized solution finishes this one -- plain backtracking would
# have to look at more trees than there are atoms in anything you care about.
FIFTY_BIN_FREQUENCIES = (4, 3, 28, 100, 5) * 10
FIFTY_BIN_COST = 96532


def test_backtracking_finds_the_optimal_cost():
    for frequencies, rotation, move, scan, cost, tree in OPTIMAL_BST_CASES:
        assert cost == optimal_cost_backtracking(
            frequencies, cost_per_rotation=rotation, move_cost_per_distance=move, cost_per_scan=scan
        ), frequencies


def test_memoized_finds_the_same_costs():
```

```
    for frequencies, rotation, move, scan, cost, tree in OPTIMAL_BST_CASES:
        assert cost == optimal_cost_memoized(
            frequencies, cost_per_rotation=rotation, move_cost_per_distance=move, cost_per_scan=scan
        ), frequencies


def test_memoized_handles_fifty_bins():
    assert FIFTY_BIN_COST == optimal_cost_memoized(
        list(FIFTY_BIN_FREQUENCIES), cost_per_rotation=3, move_cost_per_distance=1, cost_per_scan=2)


def test_optimal_tree_and_its_cost():
    for frequencies, rotation, move, scan, cost, tree in OPTIMAL_BST_CASES:
        assert (cost, tree) == optimal_tree(
            frequencies, cost_per_rotation=rotation, move_cost_per_distance=move, cost_per_scan=scan
        ), frequencies


def test_optimal_tree_still_works_on_fifty_bins():
    frequencies = list(FIFTY_BIN_FREQUENCIES)
    cost, tree = optimal_tree(
        frequencies, cost_per_rotation=3, move_cost_per_distance=1, cost_per_scan=2)
    assert FIFTY_BIN_COST == cost
    # every bin appears exactly once, and the tree is a search tree on the bin numbers
    def bins_in_order(t):
        return [] if t == () else bins_in_order(t[1]) + [t[0]] + bins_in_order(t[2])
    assert list(range(len(frequencies))) == bins_in_order(tree)


if __name__ == "__main__":
    # Runs the tests without pytest, so you can step through them in a debugger.
    for test in [test_backtracking_finds_the_optimal_cost,
                 test_memoized_finds_the_same_costs,
                 test_memoized_handles_fifty_bins,
                 test_optimal_tree_and_its_cost,
                 test_optimal_tree_still_works_on_fifty_bins]:
        test()
        print("passed:", test.__name__)
```

End of the professor's provided test code.

Part 5 -- Submission and Evaluation

Make sure that your names and the date are included in comments at the top of your .py file.

Screenshots are how parts 2, 3, and 4 get graded, so take one per part, each showing that part's tests passing along with the command you ran.

Submit:

Your .py file

The three screenshots, labeled by part

A brief paragraph (this can be a canvas comment) describing how the assignment went, how well you (and your partner) accomplished each of the 4 parts.

Rubric for Assignment: Group Project - Optimal Binary Search Tree (1)

Criteria	Points
good answers for the questions in part 1 (other parts should be straigh-forward given your answers)	10
part 2 - correct and clean backtracking implementation, with a screenshot of its tests passing	5
part 3 - correct and clean dynamic programming implementation, with a screenshot of its tests passing, including the fifty-bin case	4
part 4 - correct and clean dynamic programming implementation that returns the optimal tree, with a screenshot of its tests passing	4

satisfactory reflection	2

Each criterion is Full Marks (the points listed) or No Marks (0 pts).

3. Cross-check: professor's summary vs. the book

Each row quotes the professor's claim (left), the matching book text (middle), and whether they agree (right). "Added" means the book does not say it, so it is the professor's own clarification. Verify the middle column against the scans in Section 1.

Professor's summary (Canvas)	Book (Problem 40)	Verdict
"The problem describes a library where books are fetched by a robot."	Online bookstore DeNile; warehouse robots retrieve books from bins.	Reframed (library vs. warehouse). Same model.
"Books are stored in bins ordered by increasing ISBN---one bin per owned book title"	"a long row of bins, each containing multiple copies of a single book. These bins are arranged in sorted order by ISBN"	Match.
"books are requested, one copy at a time ... which then follows a binary search tree over the bin indexes"	"the robot must first find that book's bin using a binary search"; "compute a binary search tree over the bins"	Match.
"expected number of times each bin will be accessed on any particular day ... An array of expected bin frequencies"	"an array f[1..n] of integers, where f[i] is the number of times that the robot will be asked to retrieve a book from the ith bin"	Match. Book indexes from 1; the assignment is 0-based.
"The robot must start out at bin 0"	"The robot always starts at the '0th bin' (where the books are loaded into boxes to ship to customers)."	Match on start. See flag A below.
"facing in the 'negative' direction (toward the front desk)"	Not stated.	ADDED by professor.
"the robot can only move in the direction it is facing. Whenever the robot must change directions it incurs a cost, and the cost_per_rotation"	"Reversing the robot's direction of motion ... requires γ additional seconds"	Match. γ = cost_per_rotation. "Can only move the way it faces" is implied, not written, in the book.
"a target index which incurs an additional movement cost which is proportional to the distance ... move_cost_per_distance"	"Moving the robot from the ith bin to the jth bin requires α|i − j| seconds"	Match. α = move_cost_per_distance.
"scans the QR-code on the bin ... less than, equal to, or greater ... cost_per_scan"	"Reading an ISBN requires β seconds"; robot "must be directly in front of a bin" to read it.	Match. β = cost_per_scan. QR-code and less/equal/greater wording are the professor's detail.
"The robot can scan while facing in the negative or positive direction."	Not stated.	ADDED by professor (scanning never forces a turn).
"If the robot discovers that the scanned bin has the target ISBN, the robot ensures that it is facing in the negative direction and then moves back to index 0"	"When the robot finds the target bin, it extracts one book from that bin and returns to 'the 0th bin'."	Book says return to bin 0; the facing-negative requirement (so a possible rotation cost) is ADDED.
"find ... the binary search tree (over bin indexes) that minimizes the total expected cost of all book deliveries during the day"	"compute a binary search tree over the bins that minimizes the total time the robot spends searching for books"	Wording differs: "searching" (book) vs. "deliveries" (professor). See flag B.
"given a list of item expected frequencies and the three cost parameters"	"array f[1..n] ... along with the time parameters α, β, and γ"	Match.

Symbol map

α (book) = move_cost_per_distance | β = cost_per_scan | γ = cost_per_rotation | $f[1..n]$ = frequencies (0-indexed in Python)

Flags worth knowing about

A. Where bin 0 is. In the book, the "0th bin" is the loading spot and the book bins are $f[1..n]$. In the assignment, the bins are indexed 0, 1, 2, ... and the robot starts at bin 0. I checked this against the first provided test: frequencies [5], rotation 3, move 1, scan 2 gives cost 10 = 5 × one scan (2), with no move or rotation cost. That fits the assignment's reading (start position = bin 0). Use the assignment's model; the book's is background.

B. What counts as cost. The book says "time the robot spends searching," which could be read as excluding the trip back to bin 0. The professor's summary explicitly includes the trip back (turning to face negative, then moving to 0), and the provided tests are the final word. Part 1 asks you to hand-verify the first three test cases for exactly this reason.

C. Only in the professor's version. Starting direction (negative), scanning in either direction, the four-part breakdown, the required function and parameter names, and "recreate on paper without other resources".

D. Only in the book. "Design and analyze": the book asks for a running-time analysis, while the Canvas parts ask for code plus Part 1 written answers.

4. Our repo: SamAndJoelAttemptRoboBST

Link: https://github.com/samuelschuetz77/SamAndJoelAttemptRoboBST (public, owned by samuelschuetz77, default branch *master*)

It is a clone of Harry's repo (dharryc/optBinSearchTree) with its commit history kept, renamed to SamAndJoelAttemptRoboBST (project and package name), plus a README and a file of earlier BST code.

What we have started on

- **Project setup:** a `uv` project on Python 3.14 with the package skeleton (Harry).
- **description.md:** the start of a plain-English description of the problem and the three costs (Harry).
- **main.py:** the eight provided test cases, the fifty-bin case and the five test functions pasted in. They call `optimal_cost_backtracking`, `optimal_cost_memoized` and `optimal_tree`, which do not exist yet, so the tests cannot pass today.
- **how_robot_works.md:** a worked 7-bin walkthrough of one day's cost: the rules, the shelf, the tree, the cost of each request, where rotations happen, why this tree wins, and how it was checked (Sam).
- **previous_bsts.py:** Sam's earlier optimal-BST code in one file, with each version explained (see below). Its tests pass.
- **Hand-checking:** the first test case ([5] costs 10) is worked by hand. [4, 3] and [4, 30] are next.

Not done yet

- Part 1 written answers (function name and parameters, the subproblem and its parameters, base cases, choice of root and how costs combine).
- Part 2 backtracking, Part 3 memoized version (must handle fifty bins), Part 4 returning the optimal tree.
- Three test-pass screenshots (one per part) and the reflection paragraph.
- Open question from the log: during a search, does each left or right step need another rotation? No example has been hand-checked yet.

The earlier BSTs (previous_bsts.py)

These come from the "Backtracking for Optimal Binary Search Trees" assignment. All four share one idea: try every key in the range as the root, solve the left and right ranges recursively, add the range's total frequency (every search passes this root), and keep the cheapest. An empty range costs 0. There is no memoization, so they are exponential; the robot project's Part 3 is where caching fixes that.

Function	How it works
opt_cost(start, end, frequencies)	Plain backtracking, cost only. Passes `frequencies` down every recursive call.
opt_cost_part2(frequencies)	Same algorithm with a nested `solve(start, end)` that reads `frequencies` from the outer function. A subproblem is now just two ints, which is what a cache key needs.
opt_cost_and_tree(keys, frequencies)	solve returns a (cost, tree) pair. A tree is the nested tuple (key, left, right); None is empty. It keeps the tree that goes with the cheapest cost, with no change in running time.
opt_cost_and_tree_with_l(keys, frequencies, L)	Adds a penalty L each time a search goes left. Choosing a root adds L × sum(frequencies[start:root]), the searches that turn left here. With L=3 on [2,1,1,2,3] the best tree changes shape (cost 28 vs. 19 with no penalty).

Note: the file has four functions, not three, because Parts 1 and 2 of that assignment were the same algorithm before and after the refactor.

5. Learning log: questions and insights on the robot BST

Summarized from *learning log.md* (entries from 9/20 to 10/5). Wording is condensed from your own entries; confidence values are yours.

9/20/2026, Log 7: What is an optimal BST optimizing?

Question: What is an optimal BST actually optimizing, and why do key frequency, root choice and the left/right subproblems change total cost?

Insights: The task is to design one tree for many future searches, not to search for one key. Each key appears once; frequency is how often it is searched. The sorted order is fixed and only the shape is open. "Somewhere in the left subtree" does not mean a single left edge. Solve it by trying every root, splitting into left and right subproblems, and keeping the cheapest.

Cost: frequency × comparisons; a frequent key deep in the tree is paid for over and over. Recurrence: best left + best right + sum of frequencies in the range. Anchor example A B C D E with frequencies 2 1 1 2 3 has optimal cost 19. The best tree is not necessarily the balanced-looking one.

Meta: tree topics need a visual anchor for you first; voice-only did not work. (Hours 1, confidence 4.)

10/3/2026, Log 9: Best route to Friday's robot-BST project

Question: What is the best route to Friday's project and Monday's log?

Insights: Combine project work with targeted reading. Start by hand-checking the three smallest provided robot cases to validate the cost model. Use JEA 3.4 and 3.9 for the subproblem questions and 3.1 and 3.6-3.8 to compare states, base cases and choices. Finish backtracking, memoization and tree reconstruction before Friday, one passing-test screenshot per part. (Confidence 5.)

10/5/2026, Log 9 (call with Harry): How does the BST relate to the flat shelf of bins?

Question: How does a BST relate to a flat shelf of bins? Why jump past bins without scanning? Is reaching the root a climb that could need direction changes?

Insights: The shelf is a flat array and the BST is only instructions for which bin to check next. Directions mean toward or away from bin 0, not up or down tree levels; left/right in the tree means facing negative/positive on the shelf. The robot can pass bins without scanning; it scans only at tree nodes, root first. Skipping a layer saves scan cost but not movement cost, because distance is in bin indexes. You kept picturing the robot climbing the tree until Harry corrected it. (Confidence 5.)

10/5/2026, Log 9: How many rotations per delivery?

Question: How many rotations does one delivery cost? Does "move 5 units" mean 5 tree levels? Do you only move one way to reach the root?

Insights: The robot starts at bin 0 facing negative, so a root at bin 0 needs no rotation or movement. Units are bin-index distance, not tree levels. Reaching a root above bin 0 takes one rotation to face positive, then one move. After finding the book, at most one rotation is needed to face negative and return to 0.

Still open: whether each left or right step during the search needs another rotation; no example has been hand-checked. (Confidence 4.)

10/5/2026, Log 10: Why is the cost for [5] equal to 10?

Question: Why is the optimal cost for [5] equal to 10 when no target is given? Why scan the root five times? How do frequencies and the cost parameters fit in?

Insights: Frequencies are expected requests per day, and the optimal cost is the whole day's cost. Each request pays its own scan, move and rotation costs, weighted by frequency. Every request passes through the root, so the root's cost uses the span's total frequency (the spec's hint). The cost parameters are test inputs, so the optimal cost needs no target. You first thought the cost belonged to one search for one target; for one bin it is 5 × scan cost 2 = 10. (Hours and confidence not filled in yet.)

Gaps to fill before this is graded work: the Log 10 entry above has blank When Identified, Importance, Hours, Minutes and Confidence fields, and the next Log 10 entry is empty.